

Dokumentacja

Gra wyścigowa „Dodge 'em” – Artem Sukhodolia

1. Wprowadzenie

Projekt to prosta gra wyścigowa dla dwóch graczy zbudowana w **Pythonie** z wykorzystaniem biblioteki **pygame** do renderowania i obsługi wejścia. Kod jest podzielony na sześć modułów: `Car.py`, `Game.py`, `Interfaces.py`, `Engine.py`, `Gapi.py` oraz `main.py`. Architektura projektu opiera się w dużej mierze na zasadach programowania obiektowego (OOP) -- w szczególności na enkapsulacji, abstrakcji, kompozycji oraz odwróceniu zależności.

2. Instrukcja uruchomienia:

Wymagania: Python 3.10+ oraz biblioteka `pygame`.

Kroki:

1. Skopiować wszystkie pliki projektu.
2. Otworzyć terminal w tym folderze.
3. Zainstalować bibliotekę:

```
pip install pygame
```

5. Uruchomić grę:

```
python main.py
```

Sterowanie:

- gracz 1 -- W/A/S/D + Shift (nitro)
- gracz 2 -- strzałki + Enter (nitro)
- R -- reset wyścigu
- Esc -- wyjście.

3. Ogólna architektura

Program jest zbudowany wokół czterech głównych klas oraz dwóch interfejsów abstrakcyjnych:

- `Car` -- model fizyki i stanu pojedynczego samochodu
- `PlayerState / Game` -- logika rozgrywki (silnik gry, niezależny od `pygame`)
- `Interfaces` (`IInputProvider`, `IRenderer`) -- abstrakcyjne kontrakty
- `Gapi` (`PyGameInput`, `PyGameRenderer`) -- konkretna implementacja oparta na **pygame**
- `Engine` -- klasa spinająca wszystko w pętlę główną gry

Kluczową ideą architektoniczną jest oddzielenie logiki gry (`Game.py`, `Car.py`) od warstwy prezentacji i wejścia (`Gapi.py`). Dzięki temu klasa `Game` nie zależy bezpośrednio od `pygame` -- komunikuje się wyłącznie poprzez abstrakcyjne interfejsy.

4. Zasady OOP zastosowane w projekcie

4.1 Abstrakcja i programowanie wobec interfejsu

Plik Interfaces.py definiuje dwie klasy abstrakcyjne (ABC) z modułu abc:

- IInputProvider -- kontrakt dla obsługi wejścia (poll_events, get_actions)
- IRenderer -- kontrakt dla renderowania (setup, render, tick)

Metody oznaczone dekoratorem **@abstractmethod** wymuszają, aby każda klasa dziedzicząca dostarczyła własną implementację. Jest to klasyczny przykład abstrakcji -- klient (klasa Engine) nie wie nic o szczegółach implementacyjnych.

```
class IRenderer(ABC):
    @abstractmethod
    def setup(self, game: "Game") -> None: ...

    @abstractmethod
    def render(self, game: "Game") -> None: ...

    @abstractmethod
    def tick(self) -> float: ...
```

4.2 Odwrócenie zależności (Dependency Inversion)

Klasa Engine nie tworzy sama obiektów renderera ani providera wejścia -- otrzymuje je w konstruktorze:

```
class Engine:
    def __init__(self, game: Game, renderer: IRenderer, input_provider: IInputProvider):
        self._game = game
        self._renderer = renderer
        self._input_provider = input_provider
```

Dzięki temu Engine operuje wyłącznie na abstrakcjach (IRenderer, IInputProvider), a nie na konkretnych klasach z pygame. Pozwala to np. podmienić warstwę graficzną (np. na inną bibliotekę) bez modyfikowania logiki silnika gry -- jest to zgodne z zasadą D z SOLID.

4.3 Dziedziczenie

Konkretne klasy PyGameInput i PyGameRenderer dziedziczą po interfejsach abstrakcyjnych i implementują wszystkie wymagane metody:

```
class PyGameInput(IInputProvider): ...
class PyGameRenderer(IRenderer): ...
```

4.4 Enkapsulacja

Klasy chronią swój wewnętrzny stan, udostępniając kontrolowany dostęp:

- PlayerState przechowuje prywatne pole `_away` oraz `_start` -- nie są one bezpośrednio modyfikowane z zewnątrz.
- Game przechowuje prywatne pole `_is_wall` oraz metodę `_check_wall`, dostępną tylko wewnętrznie.
- Stan jest zmieniany wyłącznie poprzez publiczne metody, takie jak `update()`, `reset()` czy `set_wall_checker()`.

Dodatkowo, każda kluczowa klasa (Car, PlayerState, Game, PyGameInput, PyGameRenderer) jawnie blokuje kopiowanie poprzez nadpisanie metod `__copy__` i `__deepcopy__`, rzucając wyjątek `TypeError`.

```
def __copy__(self):
    raise TypeError("Car does not support copying")
```

4.5 Kompozycja

Zamiast dziedziczenia, do budowania złożonych obiektów wykorzystywana jest kompozycja:

- PlayerState zawiera obiekt Car
- Game zawiera dwa obiekty PlayerState (p1, p2)
- Gapi jest fasadą łączącą PyGameRenderer i PyGameInput w jeden spójny obiekt

Taka struktura pozwala łatwo testować i ponownie wykorzystywać poszczególne komponenty niezależnie od siebie.

4.6 Polimorfizm

Dzięki wspólnemu interfejsowi IRenderer/IInputProvider, klasa Engine może operować na dowolnej implementacji tych interfejsów w sposób polimorficzny -- wywołanie `renderer.render(game)` działa identycznie niezależnie od konkretnej klasy renderera, o ile implementuje ona interfejs IRenderer.

5. Opis poszczególnych modułów

5.1 Car.py

Zawiera dwie klasy:

- MathVector2 -- prosta klasa wektora 2D z operacjami matematycznymi (dodawanie, odejmowanie, mnożenie, interpolacja liniowa). Implementuje przeciążanie operatorów (`__add__`, `__sub__`, `__mul__`, `__rmul__`), co jest przykładem polimorfizmu operatorowego. Jest to odpowiednik Vector2 z biblioteki pygame, stworzony po to, aby logika nie była zależna od pygame.
- Car -- reprezentuje fizykę pojazdu: pozycję, prędkość, kąt obrotu. Metoda `update()` oblicza nową pozycję na podstawie wejścia gracza.

5.2 Game.py

Zawiera klasy PlayerState oraz Game, które stanowią rdzeń logiki gry, całkowicie niezależny od silnika graficznego. PlayerState śledzi liczbę okrążeń i stan samochodu, natomiast Game koordynuje aktualizację obu graczy, wykrywanie kolizji ze ścianami oraz przekraczanie linii mety.

5.3 Interfaces.py

Definiuje abstrakcyjne kontrakty IInputProvider i IRenderer, które oddzielają logikę gry od konkretnej technologii renderowania.

5.4 Gapi.py

Konkretna implementacja warstwy graficznej oparta na pygame. Zawiera PyGameInput (obsługa klawiatury), PyGameRenderer (rysowanie toru, samochodów, interfejsu HUD) oraz klasę-fasadę Gapi, łączącą oba komponenty.

5.5 Engine.py

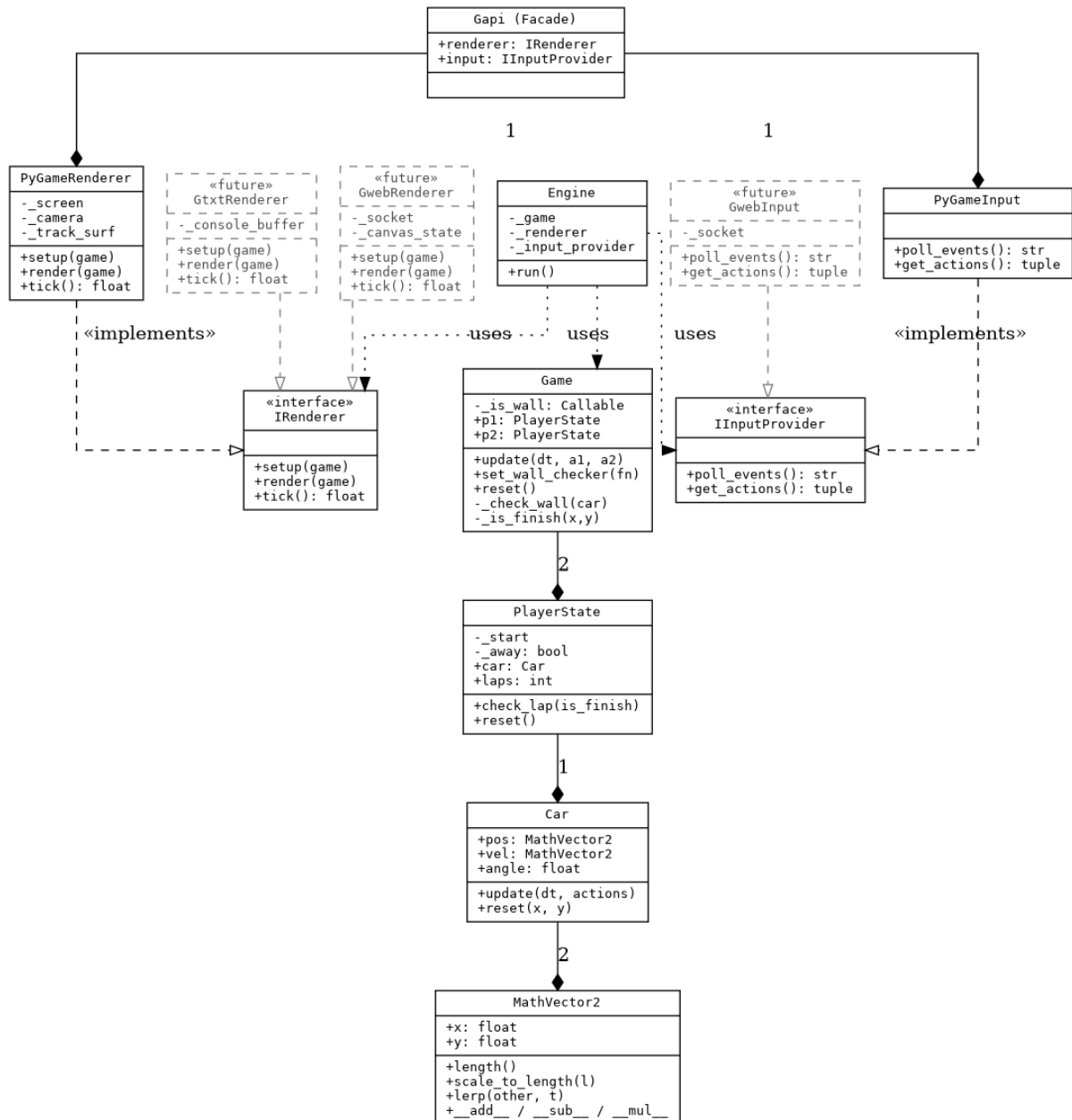
Klasa Engine implementuje główną pętlę gry (game loop): pobiera zdarzenia, aktualizuje stan gry i renderuje klatkę, korzystając wyłącznie z abstrakcyjnych interfejsów.

5.6 main.py

Punkt wejścia programu -- tworzy konkretne instancje (Gapi, Game), a następnie wstrzykuje je do Engine i uruchamia grę.

6. Diagram klas (UML)

Diagram przedstawia wszystkie klasy projektu, relacje implementacji interfejsów, kompozycji oraz wstrzykiwania zależności. Elementy przerywane, szare, oznaczone „future” to klasy, które nie istnieją jeszcze w kodzie -- dodano je, aby zademonstrować, jak łatwo architekturę można rozszerzyć o nowe sposoby wyświetlania gry, nie naruszając istniejącej logiki.



Legenda:

- Strzałka z pustym trójkątnym grotem i linią przerywaną -- implementacja interfejsu.
- Romb -- kompozycja (np. Game posiada 2× PlayerState).
- Linia kropkowana „uses” -- zależność poprzez wstrzykiwanie w konstruktorze.
- Elementy szare -- klasy potencjalne, pokazujące otwartość architektury na rozszerzeni

6.1 Klasy potencjalne: GtxtRenderer oraz GwebRenderer/GwebInput

Aktualnie jedyną konkretną implementacją interfejsów IRenderer i IInputProvider jest warstwa oparta na pygame (Gapi.py). Ponieważ Game i Engine nie znają żadnych szczegółów pygame -- a jedynie kontrakty z Interfaces.py -- można dodać inne sposoby wyświetlania gry, nie zmieniając ani jednej linijki w Car.py, Game.py czy Engine.py. Na diagramie zaznaczono dwa przykłady:

- GtxtRenderer / GtxtInput -- prosta implementacja konsolowa, np. do debugu.
- GwebRenderer / GwebInput -- implementacja webowa, np. renderowanie w przeglądarce.

Jest to demonstracja zasady Open/Closed (SOLID) oraz odwrócenia zależności -- moduł wysokiego poziomu (Game, Engine) jest całkowicie niezależny od modułów niskiego poziomu.

6.2 Co należy zmienić w kodzie, aby dodać wyjście na Gweb / Gtxt

Żadna z poniższych zmian nie dotyczy Car.py, Game.py, Interfaces.py ani Engine.py -- to kluczowa zaleta obecnej architektury.

Krok 1 -- utworzyć nowy plik, np. Gtxt.py lub Gweb.py

Nowe klasy muszą dziedziczyć po IRenderer i IInputProvider oraz implementować wszystkie metody abstrakcyjne:

```
from Interfaces import IInputProvider, IRenderer

class GtxtRenderer(IRenderer):
    def setup(self, game) -> None:
        pass # np. przygotowanie bufora konsoli

    def render(self, game) -> None:
        # np. print(f"P1: {game.p1.car.pos.x:.0f},{game.p1.car.pos.y:.0f}")
        pass

    def tick(self) -> float:
        return 1 / 60 # stały krok czasu

class GtxtInput(IInputProvider):
    def poll_events(self) -> str:
        return "continue" # np. odczyt z stdin

    def get_actions(self):
        a1 = {"forward": False, "brake": False, "left": False, "right": False, "nitro":
False}
        a2 = dict(a1)
        return a1, a2
```

Krok 2 -- dodać fasadę analogiczną do Gapi

```
class Gtxt:
    def __init__(self):
        self.renderer = GtxtRenderer()
        self.input = GtxtInput()
```

Krok 3 -- podmienić jedną linię w main.py

main.py jest jedynym miejscem w całym projekcie, które trzeba zmienić, aby przełączyć się na nową implementację:

```
from Engine import Engine
from Game import Game
from Gtxt import Gtxt # zamiast: from Gapi import Gapi

if __name__ == "__main__":
    api = Gtxt() # zamiast: api = Gapi()
    game = Game()
    Engine(game, api.renderer, api.input).run()
```

Dzięki temu, że Engine przyjmuje renderer i input_provider jako abstrakcje (IRenderer, IInputProvider) poprzez konstruktor, a nie tworzy ich samodzielnie, cała reszta systemu (Game, Car, Engine) działa bez żadnych modyfikacji -- wystarczy podmienić jeden import i jedną linię tworzącą obiekt fasady.

7. Zasady SOLID w projekcie

Zasada	Klasa	Jak zrealizowana w projekcie
S — Single Responsibility	Car, Game, PyGameRenderer, PyGameInput	Car odpowiada wyłącznie za fizykę ruchu; Game -- tylko za logikę rozgrywki; PyGameRenderer -- wyłącznie za rysowanie; PyGameInput -- wyłącznie za odczyt klawiatury.
O — Open/Closed	IRenderer, IInputProvider	Nowy sposób wyświetlania gry (np. GtxtRenderer, GwebRenderer) można dodać implementując interfejs, bez modyfikacji Engine ani Game -- kod jest otwarty na rozszerzenie, zamknięty na modyfikację.
L — Liskov Substitution	PyGameRenderer / PyGameInput	Każda klasa implementująca IRenderer lub IInputProvider może zastąpić inną w Engine bez zmiany zachowania pętli gry -- Engine wywołuje te same metody <u>niezależnie</u> od konkretnej klasy.
I — Interface Segregation	IRenderer, IInputProvider	Interfejsy są wąskie i skupione: IRenderer ma tylko metody rysowania, IInputProvider tylko metody wejścia -- klasy nie muszą implementować zbędnych metod.
D — Dependency Inversion	Engine	Engine zależy od abstrakcji (IRenderer, IInputProvider), a nie od konkretnych klas PyGameRenderer/PyGameInput. Konkretnie implementacje są wstrzykiwane przez konstruktor (main.py).